

CURSO DE ENGENHARIA DE SOFTWARE

Disciplina de Laboratório de Programação

Aula 08: Strings em C

Prof. Me. Ivandro José de Freitas Rocha

Goianésia – 2026

Compreender o conceito de strings em C como vetores de caracteres terminados por '\0'; declarar e inicializar strings; utilizar as funções de entrada e saída; aplicar as funções de manipulação, comparação, tamanho, conversão e pesquisa de strings.

CONTEÚDOS

- Strings: conceito, representação em memória e terminador '\0'
- Inicialização de strings
- Entrada e saída de strings
- Manipulação, comparação e tamanho de strings
- Conversão de caixa e conversão numérica
- Pesquisa em strings.

REFERÊNCIAS

- DEITEL & DEITEL. C: como programar. 6. ed. (Cap. 8)
- SCHILDT. C completo e total. 3. ed. (Cap. 5)

CONCEITO

São sequências de caracteres representadas entre aspas.

Em C não existe um tipo `STRING` explícito, não há palavra reservada que declare uma variável de string, como acontece em Pascal ou Java.

Para armazenar uma string, utiliza-se um vetor ou matriz do tipo `char`, e o fim da cadeia é marcado pelo caractere terminador `'\0'` (null zero).



Caracteres como Blocos Fundamentais

Caracteres são blocos de montagem fundamentais dos programas-fonte. Cada programa é composto por uma sequência de caracteres que, ao serem agrupados de modo significativo, são interpretados pelo computador como uma série de instruções que serão usadas na realização de tarefa.

Constante de Caracter

É um valor INT representado por um caracter entre aspas simples.

Conjunto de Caracteres da Máquina

Exemplo: o código ASCII.

STRING

Consistem em uma série de caracteres tratados como uma única entidade.

Caracteres Especiais

Acentos, pontuação, etc.

Strings Literais ou Constantes Strings

São escritas entre aspas duplas.

EXEMPLOS:

```
"João da Silva"
```

```
"14-2107-7000"
```

É um array de caracteres que termina no caractere '\0'

O valor de uma STRING é o endereço de seu primeiro caractere

STRINGs são acessadas por meio de um ponteiro para o primeiro caractere da STRING

EXEMPLOS:

```
char cor[] = "azul";  
const char *corPtr = "azul";  
char cor[] = {'a', 'z', 'u', 'l', '\0'}
```

INICIALIZANDO STRINGS

Existem duas formas principais de inicializar strings em C:

```
char nome[] = {'E', '\0'};  
char nome[] = "Elaine";
```

A segunda forma é mais comum e prática. O compilador adiciona automaticamente o caractere nulo `'\0'` ao final.

```
#include <stdio.h>  
#include <stdlib.h>  
main(){  
    char saudacao[]="Saudacoes, ";  
    char nome[80];  
    printf("Digite o seu nome: ");  
    gets(nome);  
    printf("%s%s \n",saudacao,nome);  
    system("PAUSE");  
}
```

Saída do programa

```
Digite o seu nome: Elaine Cecilia Gatto  
Saudacoes, Elaine Cecilia Gatto  
Pressione qualquer tecla para continuar...
```

STRINGS CONSTANTES NA MEMÓRIA

Sempre que o compilador encontra qualquer coisa entre aspas duplas, ele reconhece que se trata de uma string constante.

EXEMPLO:

```
printf("%s", "Saudações");
```

 Cada caractere ocupa uma posição de memória sequencial. O terminador `\0` marca o fim da string.

Posição da Memória	String
1092	S
1093	a
1094	u
1095	d
1096	a
1097	ç
1098	õ
1099	e
1100	s
1101	\0

SCANF()

É uma função da biblioteca padrão de entrada/saída (<stdio.h>) que lê caracteres até encontrar um espaço, uma tabulação, uma nova linha ou o fim do arquivo.

Use %s de modo que scanf leia a quantidade de caracteres exatos que você deseja.

EXEMPLO:

```
char word[20];  
scanf("%19s", word);
```



Sempre limite o número de caracteres lidos com scanf para evitar estouro de buffer.

SCANF()

```
#include <stdio.h>
#include <stdlib.h>
main(){
    char nome[100];
    printf("Digite o seu nome: ");
    scanf("%99s", nome);
    printf("Saudacoes, %s\n", nome);
    system("PAUSE");
}
```

Saída do programa

```
Digite o seu nome: Elaine Cecilia Gatto
Saudacoes, Elaine
Pressione qualquer tecla para continuar...
```

Observe a saída do programa!
Consegue notar o que há de diferente entre a entrada de dados e a saída?

SCANF()

```
#include <stdio.h>
#include <stdlib.h>
main(){
    char nome[100];
    printf("Digite o seu nome: ");
    scanf("%99s", nome);
    printf("Saudacoes, %s\n", nome);
    system("PAUSE");
}
```

Saída do programa

```
Digite o seu nome: Elaine Cecilia Gatto
Saudacoes, Elaine
Pressione qualquer tecla para continuar...
```

- A instrução lê cada caractere digitado e o armazena a partir do endereço nome, terminando quando encontra um caractere branco, nesse ponto, o '\0' é incluído automaticamente na próxima posição livre.
- Como o espaço em branco marca o fim da entrada, apenas o primeiro nome digitado aparece na saída.
- Não é permitido ultrapassar o limite do vetor, e o espaço para o '\0' deve ser previsto.

SCANF()

```
#include <stdio.h>
#include <stdlib.h>
main(){
    char nome[100];
    printf("Digite o seu nome: ");
    scanf("%99s", nome);
    printf("Saudacoes, %s\n", nome);
    system("PAUSE");
}
```

Saída do programa

```
Digite o seu nome: ElaineCeciliaGatto
Saudacoes, ElaineCeciliaGatto
Pressione qualquer tecla para continuar...
```

- Mas se digitarmos o nome inteiro sem espaços, observe a saída!
- Observe esta instrução novamente. Porque está sendo usado nome e não &nome?
- Neste caso, nome é equivalente a &nome[0]
- O nome de uma matriz é o seu endereço inicial
- Scanf() é muito utilizada para ler uma mistura de tipos de dados numa mesma instrução.

FGETS()

Assinatura da Função

```
char *fgets (char *s, int n, FILE *stream);
```

Insere caracteres de fluxo especificado para o array S até que:

- um caractere de **newline seja lido**, ou
- **fim de arquivo** seja encontrado, ou
- até que **n-1 bytes sejam lidos**.

Um caractere nulo de finalização é anexado ao array e a string lida é retornada.

FGETS()

```
#include <stdio.h>
#include <stdlib.h>
main(){
    char nome[15];
    printf("Digite o seu nome: ");
    fgets(nome, 15, stdin);
    printf("Saudacoes, %s\n", nome);
    system("PAUSE");
}
```

- O código ao lado utiliza fgets. O vetor declarado tem 15 posições e somente essas 15 posições são preenchidas.
- Observa-se na saída que o nome digitado é maior que as 15 posições, e portanto, o restante do nome é desprezado.

 STDIN: fluxo de entrada-padrão.

Saída do programa

```
Digite o seu nome: Elaine Cecilia Gatto
Saudacoes, Elaine Cecilia
Pressione qualquer tecla para continuar...
```

GETCHAR()

Assinatura da Função

```
int getchar (void);
```

Insere o caractere seguinte da entrada-padrão e o retorna como um inteiro

A função `getchar()` lê um caracter e retorna um inteiro que é:

- o código do caracter, ou
- o valor EOF que corresponde a fim de ficheiro.

Entrada padrão

`getchar()` lê

Retorna inteiro

GETCHAR()

```
// Exemplo da utilização de getchar
#include <stdio.h>
#include <stdlib.h>
main(){
    char character;
    printf("Utilizando getchar()\n\n");
    printf("Entre com um caracter: ");
    fflush(stdout);
    character = getchar();
    printf("\n Você digitou o caracter %c \n", character);
    system("PAUSE");
}
```

GETCHAR()

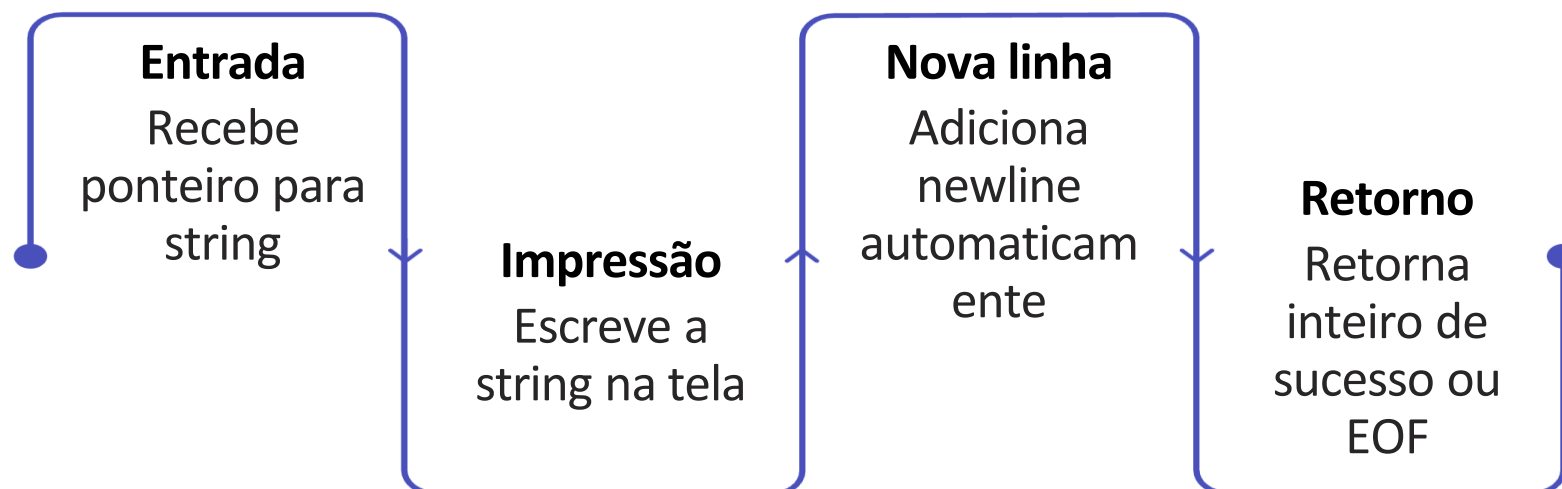
```
// Uso de getchar() em um laço para contar ocorrências do caractere 'a' na entrada
#include <stdio.h>
#include <stdlib.h>
main(){
    int c;
    int i=0;
    while((c = getchar()) != EOF)
        if (c == 'a')
            i++;
    printf("Foram lidos %d a\'s", i);
    system("PAUSE");
}
```

PUTS()

Assinatura da Função

```
int puts (const char *s);
```

Imprime a string S seguida por um caractere de newline.
Retorna um inteiro diferente de zero se for bem sucedida.
Retorna um EOF se ocorrer um erro.



PUTS()

```
#include <stdio.h>
#include <stdlib.h>
main(){
    char nome[80];
    printf("Digite o seu nome: ");
    fgets(nome);
    puts("");
    printf("Saudacoes, ");
    puts(nome);
    printf("Imprime a partir da 4ª
posicao: ");
    puts(&nome[4]);
    puts("a funcao puts() pula linha
automaticamente!");
    system("pause");
}
```

- A função puts() pula linha automaticamente!
- É o complemento da função gets()
- Propósito: imprimir uma única string por vez
- O endereço da string deve ser enviado para puts como argumento

Saída do programa

Digite o seu nome: Elaine Cecilia Gatto

Saudacoes, Elaine Cecilia Gatto

Imprime a partir da 4ª posicao: ne Cecilia Gatto
a funcao puts() pula linha automaticamente!

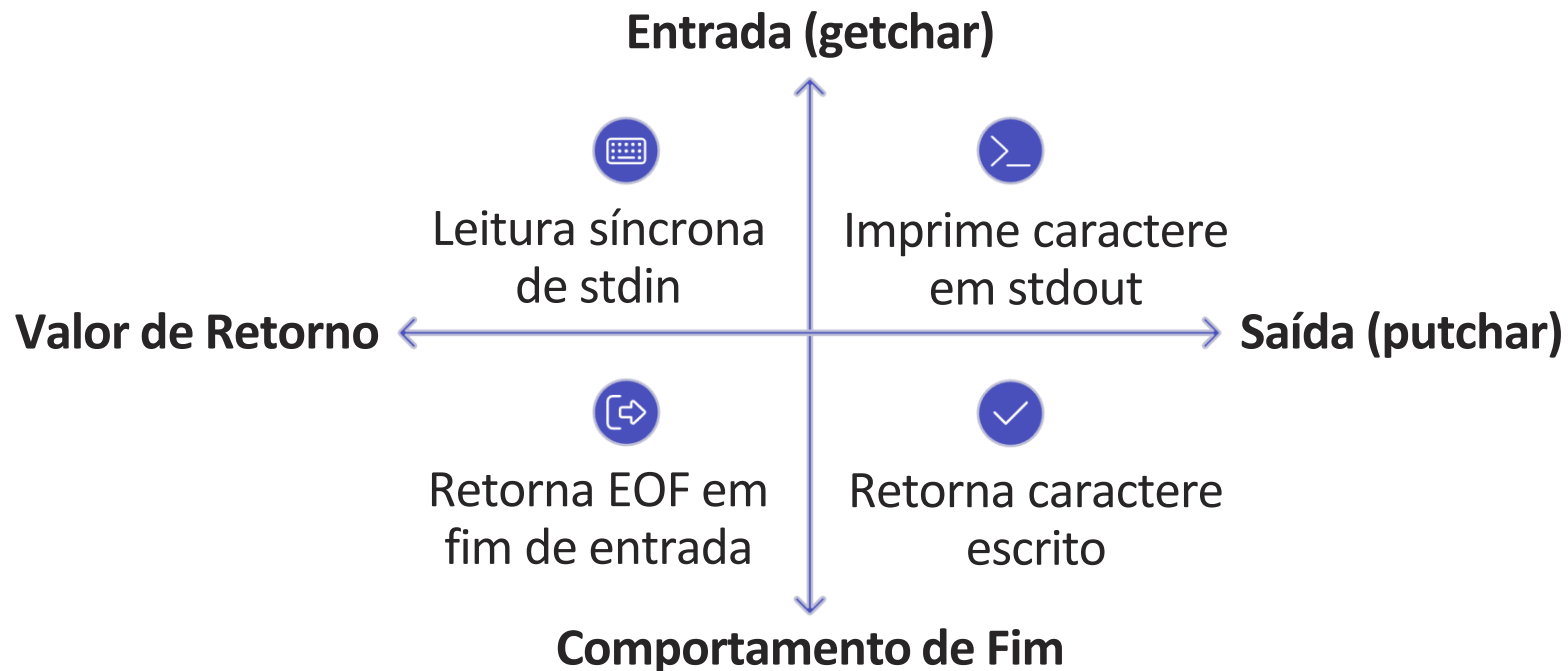
Pressione qualquer tecla para continuar...

PUTCHAR()

Assinatura da Função

```
int putchar (int c);
```

Imprime o caracter armazenado em C e o retorna como um inteiro



SNPRINTF()

Assinatura da Função

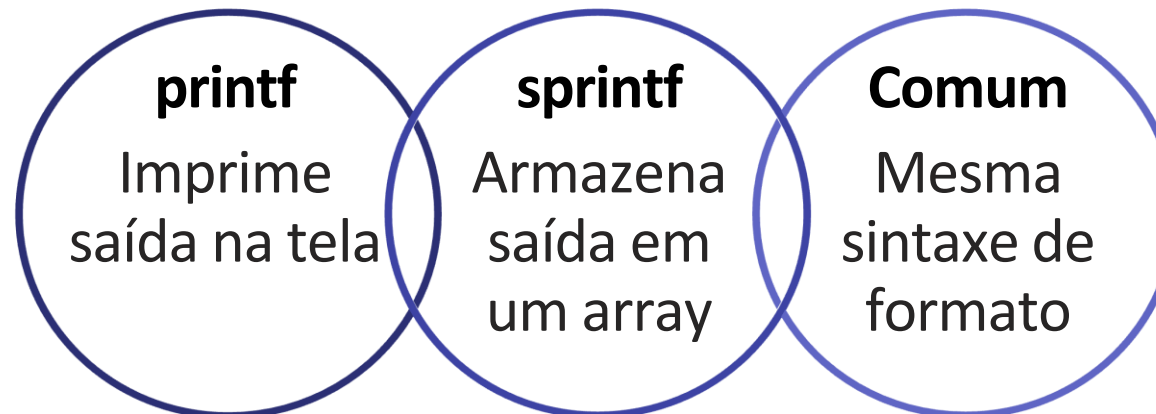
```
int snprintf (char *s, const char *format, ...);
```

Equivalente a printf.

A saída é armazenada no array S em vez de impressa na tela.

Retorna o número de caracteres escritos em S.

Retorna EOF se ocorrer um erro.



SNPRINTF()

```
#include <stdio.h>
#include <stdlib.h>
main(){
    int i;
    char string1[20];
    printf("Entre um valor inteiro: ");
    fflush(stdout);
    scanf("%d", &i);
    snprintf(string1, "Valor de i = %d", i);
    puts(string1);
    system("PAUSE");
}
```

Resumo das Funções de String em C

Entrada

scanf(), sscanf(),
fgets(),
getchar()

Saída

printf(),
sprintf(), puts(),
putchar()

FGETS() E PUTCHAR()

```
#include <stdio.h>
#include <stdlib.h>
void reverse(const char* const sPtr){
    if(sPtr[0] == '\0') return;
    else{
        reverse(&sPtr[1]); //recursão
        putchar(sPtr[0]); //exibe o caractere
    }
}
main(){
    char sentenca[80];
    printf("Digite uma linha de texto: \n");
    fflush(stdout);
    fgets(sentenca, 80, stdin);
    printf("\n Linha impressa na ordem inversa
é: \n");
    reverse(sentenca);
    system("PAUSE");
}
```

- O exemplo lê uma linha de texto com `fgets()` e a imprime na ordem inversa usando recursão com `putchar()`.

fgets() Lê uma linha de texto com limite de caracteres, evitando estouro de buffer.

putchar() Exibe um único caractere na saída padrão.

Recursão A função `reverse()` chama a si mesma avançando o ponteiro até o fim da string, depois imprime ao retornar.

STRCPY()

Assinatura da Função

```
char *strcpy (char *s1, const char *s2);
```

Copia a string

Copia a string `s2` no array `s1`.

Retorno

O valor de `s1` é retornado.

Biblioteca

`<string.h>`

Copia ou atribui o conteúdo da string de origem para a string de destino. Ambas devem ter, no mínimo, o mesmo tamanho.

Sintaxe: `strcpy(string_destino, string_origem);`

STRCPY()

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main(){
    char nome_original[30];
    char nome_copia[30];
    printf("Digite um nome: ");
    gets(nome_original);
    strcpy(nome_copia, nome_original);
    printf("Nome original: ");
    puts(nome_original);
    printf("Nome copia: ");
    puts(nome_copia);
    system("PAUSE");
}
```

Saída do programa

```
Digite o seu nome: Elaine Cecilia Gatto
Nome original: Elaine Cecilia Gatto
Nome copia: Elaine Cecilia Gatto
Pressione qualquer tecla para continuar...
```

❏ Após `strcpy()`, as duas strings possuem conteúdo idêntico, mas ocupam espaços de memória distintos.

STRCAT()

Assinatura da Função

```
char *strcat (char *s1, const char *s2);
```

Acrescenta

Acrescenta a string `s2` ao array `s1`.

Sobrescreve o nulo

O primeiro caractere de `s2` sobrescreve o caractere nulo de finalização de `s1`.

Retorno

O valor de `s1` é retornado.

Biblioteca

`<string.h>`

Concatena a string de origem ao final da string de destino. A função não verifica se a primeira cadeia tem espaço para que a segunda cadeia seja adicionada ao seu final.

Sintaxe: `strcat(string_destino, string_origem);`

STRCAT()

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main(){
    char cadeia1[100];
    char cadeia2[10];
    printf("Digite primeiro nome: ");
    gets(cadeia1);
    printf("Digite segundo nome: ");
    gets(cadeia2);
    strcat(cadeia1, cadeia2);
    printf("%s", cadeia1);
    puts("");
    system("PAUSE");
}
```

Saída:

```
Digite o seu primeiro nome: Elaine
Digite o seu segundo nome: Cecilia
Elaine Cecilia
Pressione qualquer tecla para continuar.
```

⚠️ Atenção: `strcat()` não verifica o tamanho do buffer de destino. Certifique-se de que `cadeia1` tem espaço suficiente para receber a concatenação.

STRNCPY()

Assinatura da Função

```
char *strncpy (char *s1, const char *s2, size_t n);
```

Copia com limite

Copia no máximo `n` caracteres da `string2` no array `s1`.

Retorno

O valor de `s1` é retornado.

Biblioteca

`<string.h>`

STRCPY E STRNCPY

```
#include <stdio.h>
#include <string.h>
main(void){
    char x[] = "Parabéns a você";
    char y[25];
    char z[15];
    printf("%s %s \n %s %s \n",
        "A string no array X é: ", x,
        "A string no array y é: ", y,
        strcpy(y,x));
    fflush(stdout);
    strncpy(z, x, 14);
    z[14] = '\0';
    printf("A string no array Z é: %s", z);
}
```

Array X

Contém a string original: "Parabéns a você"

Array Y (strcpy)

Recebe cópia completa de X via `strcpy(y, x)`.

Array Z (strncpy)

Recebe apenas os 14 primeiros caracteres de X. O terminador `'\0'` é adicionado manualmente em `z[14]`.

STRNCAT()

Assinatura da Função

```
char *strncat (char *s1, const char *s2, size_t n);
```

Acréscimo com limite

Acrescenta no máximo `n` caracteres da string `s2` ao array `s1`.

Sobrescreve o nulo

O primeiro caractere de `s2` sobrescreve o caractere nulo de finalização de `s1`.

Retorno

O valor de `s1` é retornado.

Biblioteca

`<string.h>`

STRCMP()

Assinatura da Função

```
int strcmp (const char *s1, const char *s2);
```

Compara

Compara a string `s1` com a string `s2`.

Retorno = 0

`s1` é igual a `s2`.

Retorno < 0

`s1` é menor do que `s2`.

Retorno > 0

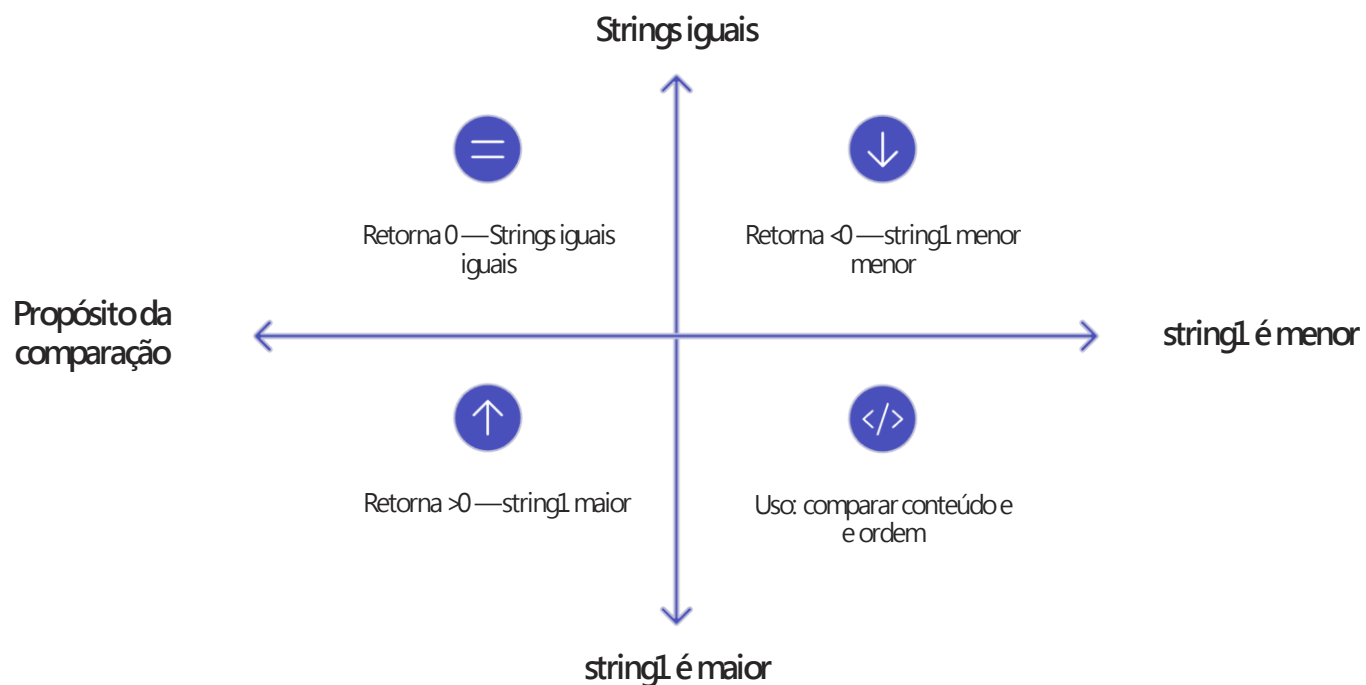
`s1` é maior do que `s2`.

Biblioteca

`<string.h>`

STRCMP()

DETALHES:



Compara conteúdos

Compara o conteúdo da string1 com o conteúdo da string da string2.



Retorna inteiro

Retorna um número inteiro que indica a relação entre as strings.



Igualdade

Se forem iguais, a função retorna **zero**.



Menor que zero

Se o número retornado for menor que zero, então a string1 é **menor** que a string 2.



Maior que zero

Se o número retornado for maior que zero, então a string1 é **maior** que a string2.

STRNCMP()

Assinatura da Função

```
int strncmp (const char *s1, const char *s2, size_t n);
```

Compara com limite

Compara até `n` caracteres da string `s1` com a string `s2`.

Retorno = 0

`s1` é igual a `s2` nos primeiros `n` caracteres.

Retorno < 0

`s1` é menor do que `s2`.

Retorno > 0

`s1` é maior do que `s2`.

É uma função de **comparação de strings**.

STRCMP()

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main(){
    char string1[20], string2[20];
    int resultado;
    printf("Digite uma palavra: ");
    gets(string1);
    gets(string2);
    resultado = strcmp(string1, string2);
    if(resultado == 0)
        puts("As strings sao iguais.");
    else
        puts("As strings sao diferentes.");
    system("PAUSE");
}
```

Leitura

Lê duas strings do usuário com `gets()`.

Comparação

Chama `strcmp(string1, string2)` e armazena o resultado em `resultado`.

Decisão

Se `resultado == 0`, as strings são iguais; caso contrário, são diferentes.

STRCMP()

Saída:

```
Digite uma palavra: oi
As strings sao iguais.
Pressione qualquer tecla para continuar.
Digite uma palavra: amanha
Digite uma palavra: hoje
As strings sao diferentes.
```

Caso 1: Strings iguais

Quando o usuário digita a mesma palavra duas vezes (ex: `oi`), `strcmp()` retorna `0` e o programa exibe *"As strings sao iguais."*

Caso 2: Strings diferentes

Quando as palavras são distintas (ex: `amanha` e `hoje`), `strcmp()` retorna valor diferente de zero e o programa exibe *"As strings sao diferentes."*

STRLEN()

Assinatura da Função

```
size_t strlen (const char *s);
```

Determina o comprimento

Determina o comprimento da string `S`.

Retorno

O número de caracteres anteriores ao caractere nulo de finalização é retornado.

Não conta o '\0'

O terminador nulo `'\0'` não é contabilizado no comprimento retornado.

Recebe como argumento o endereço de uma string e retorna o seu tamanho. O comprimento do vetor/matriz da string deve ser um a mais que o inteiro retornado por `strlen()`. O terminador `\0` não é contabilizado.

STRLEN()

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main(){
    char string1[20], string2[20];
    int tam1, tam2;
    printf("Digite uma palavra: ");
    gets(string1);
    gets(string2);
    tamanho1 = strlen(string1);
    tamanho2 = strlen(string2);
    printf("Tamanho string1: %d\n", tam1);
    printf("Tamanho string2: %d\n", tam2);
    system("pause");
}
```

Saída:

```
Digite uma palavra: teste
Digite uma palavra: testando
O tamanho da string1 e: 5
O tamanho da string2 e: 8
Pressione qualquer tecla para continuar.
```

☐ "teste" tem 5 caracteres e "testando" tem 8 caracteres. O '\0' não é contado em nenhum dos casos.

STRUPR()

```
/* Converte o conteúdo da string em
letras maiúsculas.*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main(){
    char string[20];
    printf("\n Escreva uma palavra: ");
    gets(string);
    printf("\n Original: %s", string);
    strupr(string);
    printf("\n Convertida: %s", string);
    printf("\n\n");
    system("pause");
}
```

Saída:

```
Escreva uma palavra: teste
String original: teste
String convertida: TESTE
Pressione qualquer tecla para continuar.
```

Entrada

teste (minúsculas)

Saída

TESTE (maiúsculas)

STRNLWR()

```
/* Converte o conteúdo da string em
letras minúsculas.*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main(){
    char string[20];
    printf("\n Palavra maiusculo: ");
    gets(string);
    printf("Original: %s\n", string);
    strupr(string);
    printf("Convertida: %s\n\n", string);
    system("pause");
}
```

Saída:

Escreva uma palavra com todas as letras em MAIUSCULO: TESTE
String original: TESTE
String convertida: teste
Pressione qualquer tecla para continuar.

Entrada

TESTE (maiúsculas)

Saída

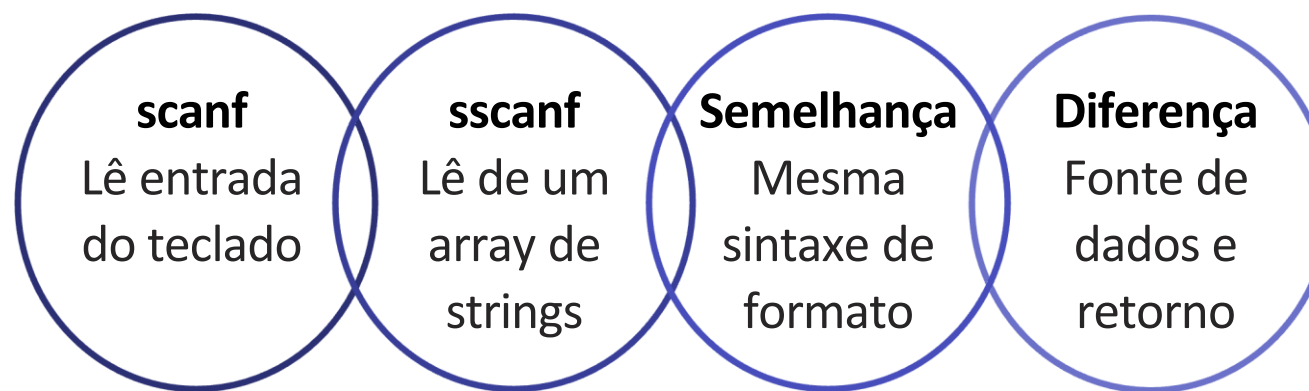
teste (minúsculas)

SSCANF()

Assinatura da Função

```
int sscanf (char *s, const char *format, ...);
```

- Equivalente a scanf.
- A entrada é lida a partir do array S em vez do teclado.
- Retorna o número de caracteres escritos em S.
- Retorna EOF (end-of-file) se ocorrer um erro.



Biblioteca de utilitários gerais: `stdlib.h`.

Convertem strings de dígitos em valores inteiros e de ponto flutuante.

- 1** **atof()**
`double atof (const char *nPtr);`
Converte a string `nPtr` em `double`.
- 2** **atoi()**
`int atoi (const char *nPtr);`
Converte a string `nPtr` em `int`.
- 3** **atol()**
`long atol (const char *nPtr);`
Converte a string `nPtr` em `long int`.
- 4** **strtod()**
`double strtod (const char *nPtr, char **endPtr);`
(*ponteiro de uma string)
- 5** **strtol()**
`long strtol (const char *nPtr, char **endPtr, int base);`
- 6** **strtoul()**
`unsigned long strtoul (const char *nPtr, char **endPtr, int base);`
Converte a string `nPtr` em `unsigned long`.

CONVERSÃO DE STRINGS — EXEMPLOS



Referência

Exemplos completos disponíveis no **Livro do Deitel**. Consulte o livro-texto para os códigos-fonte completos e análise detalhada de cada função de conversão.



Objetivo

As funções de conversão permitem transformar representações textuais de números em tipos numéricos nativos do C, essencial para processar entradas do usuário.

String → double

```
atof("3.14") → 3.14
```

String → int

```
atoi("42") → 42
```

String → long

```
atol("100000") → 100000L
```

String → unsigned long

```
strtoul("255", NULL,  
10) → 255UL
```

STRCHR()

Assinatura da Função

```
char *strchr (const char *s, int c);
```

Localiza

Localiza a primeira ocorrência do caractere **C** na string **S**.

Encontrado

Se **c** for encontrado, um ponteiro para **C** em **S** é retornado.

Não encontrado

Caso contrário, um ponteiro **NULL** é retornado.

Categoria

É uma função de **pesquisa**.



Exemplo 8.23 disponível no Livro Deitel. Consulte o livro-texto para o código-fonte completo.

STRCSPN()

Assinatura da Função

```
size_t strcspn (const char *s1, const char *s2);
```

Determina e retorna

Determina e retorna o tamanho do segmento inicial inicial da string `s1` que consiste apenas em caracteres **não** contidos na string `s2`.

Categoria

É uma função de **pesquisa**.



Exemplo 8.24 disponível no livro do Deitel. Consulte o livro-texto para o código-fonte completo e análise detalhada.

A função percorre `s1` da esquerda para a direita e conta quantos caracteres consecutivos **não** aparecem em `s2`, retornando esse comprimento como `size_t`.

STRSPN()

Assinatura da Função

```
size_t strspn (const char *s1, const char *s2);
```

Determina e retorna retorna

Determina e retorna o tamanho do segmento inicial da string `s1` que consiste em caracteres **não** contidos na string `s2`.

Categoria

É uma função de **pesquisa**.

 Exemplo 8.27 disponível no livro Deitel. Consulte o livro-texto para o código-fonte completo e análise detalhada.

Complementar a `strcspn()`: enquanto `strcspn()` conta caracteres *ausentes* de `s2`, `strspn()` conta caracteres *presentes* em `s2` *presentes* em `s2` no início de `s1`.

STRPBRK()

Assinatura da Função

```
char *strpbrk (const char *s1, const char *s2);
```

Localiza

Localiza a primeira ocorrência na string `s1` de qualquer caractere na string `s2`.

Encontrado

Se um caractere da string `s2` for encontrado, um ponteiro para caractere na string `s1` é retornado.

Não encontrado

Caso contrário, um ponteiro `NULL` é retornado.

Categoria

É uma função de **pesquisa**.



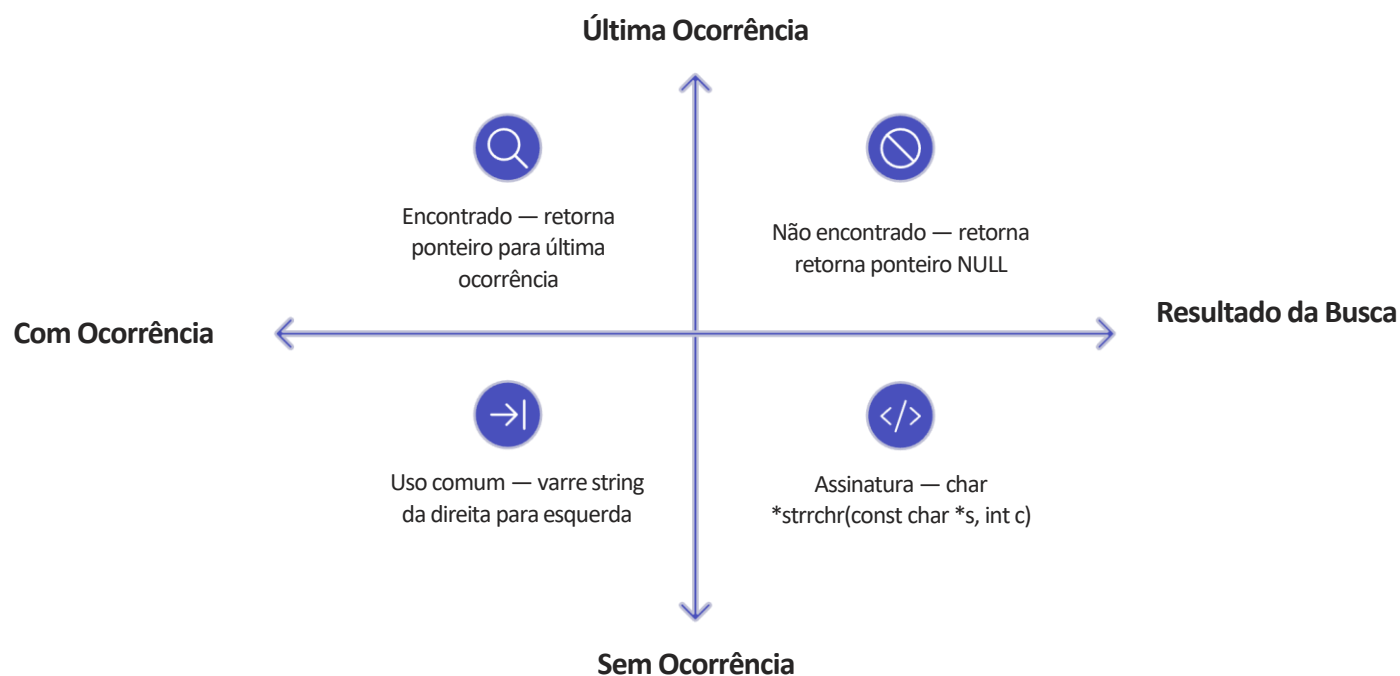
Exemplo 8.25 disponível no livro Deitel. Consulte o livro-texto para o código-fonte completo e análise detalhada.

STRRCHR()

Assinatura da Função

```
char *strrchr (const char *s, int c);
```

- Localiza a **última ocorrência** de C na string S
- Se C for encontrado, um **ponteiro para C** na string S é retornado
- Caso contrário, um **ponteiro NULL** é retornado
- É uma **função de pesquisa**
- Exemplo 8.26, livro do Deitel

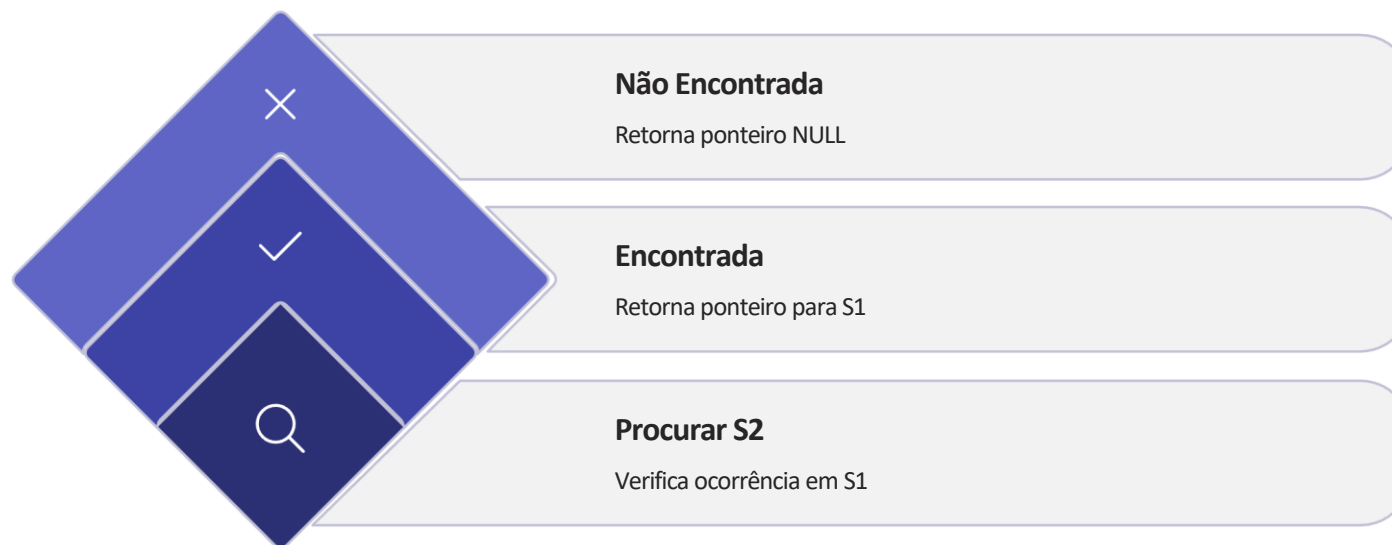


STRSTR()

Assinatura da Função

```
char *strstr (const char *s1,  
const char *s2);
```

- Localiza a **primeira ocorrência** na string S1 da string S2
- Se a string for encontrada, um **ponteiro para a string em S1** é retornado
- Caso contrário, um **ponteiro NULL** é retornado
- É uma **função de pesquisa**
- Exemplo 8.28, livro do Deitel



STRtok()

Assinatura da Função

```
char *strtok (char *s1, const char *s2);
```

1

Separação em Tokens

Uma sequência de chamadas para STRtok separa a string `s1` em tokens tokens separados por caracteres contidos na string `s2`.

2

O que são TOKENS?

Partes lógicas. Exemplo: palavras em uma linha de texto.

3

Primeira Chamada

A primeira chamada contém `s1` como primeiro argumento. Para que as chamadas seguintes continuem a separar tokens na mesma string, conter `NULL` como primeiro argumento.

4

Retorno da Função

Um ponteiro para o token em vigor é retornado por cada chamada. Se não houver mais tokens quando a função for chamada, `NULL` será retornado.

É uma **função de pesquisa**. Exemplo 8.29, livro Deitel.

EXERCÍCIOS

Desenvolver um programa que obtenha as seguintes informações para os "**n**" **colegas** da sua turma de Programação de Computadores:

Variável	Qtde Caracteres
nome	15
sobrenome	30
endereço	80
telefone	10
idade	3

Realize as seguintes tarefas:

- Copie o nome do seu colega para uma variável denominada `nome2`
- Concatene o sobrenome à variável `nome2`

EXERCÍCIOS (CONTINUAÇÃO)

1 Continuação do Exercício 1

- Obtenha o tamanho da variável `nome2`
- Imprima todas as informações de cada um dos seus colegas na tela

3 Exercício 3

Troque as letras do exercício anterior, por **minúsculas**.

2 Exercício 2

Faça um programa em C que leia 10 strings, imprima o tamanho de cada uma delas e troque as letras para **maiúsculas**. Dica: use use o `FOR`.

4 Exercício 4

Escreva um programa em C que receba o nome, a idade e o sexo de 10 pessoas e armazene, em um vetor/matriz, as pessoas que forem do **sexo masculino** e **maiores de 18 anos**.

EXERCÍCIOS (FINAL)

EXERCÍCIO 5

Escreva um programa que faça a seguinte pergunta ao usuário:

"Quem é o criador da Linguagem C?"

O usuário deve escrever a resposta e o programa deve verificar:

Resposta Correta

Uma mensagem de "**parabéns, voce acertou!**" deve ser

Resposta Incorreta

Uma mensagem de "**que pena, voce errou!**" deve ser emitida.



ivandro.rocha@uniego.edu.br

DUVIDAS?

OBRIGADO!



Prof. Me Ivandro José de Freitas Rocha